СОФИЙСКА ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО ЕЛЕКТРОНИКА „ДЖОН АТАНАСОВ“

ДЪРЖАВЕН ИЗПИТ

ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН НА ПРОФЕСИОНАЛНА КВАЛИФИКАЦИЯ – ЧАСТ ПО ТЕОРИЯ НА ПРОФЕСИЯТА

Д И П Л О М Е Н П Р О Е К Т

Тема: „……..“

професия: 481020 „Системен програмист“
специалност: 4810201 „Системно програмиране“

Дипломант: …….., 12А клас

Ръководител на дипломния проект: инж. Милена Добрикова

Подпис: ……
(дипломант)
Подпис:……
(ръководител)

София, 2026 г.

СЪДЪРЖАНИЕ

1. ВЪВЕДЕНИЕ....3
1.1. Задание за разработка....3
1.2. Анализ на заданието....3
1.3. Поставени цели....3
2. ОСНОВНА ЧАСТ....4
2.1. Избор на технологии....4
2.1.1. Избор на програмен език....4
2.2. Етапи от разработване на проекта....5
2.3. Обяснение на кода....6
3. ЗАКЛЮЧЕНИЕ....7
ИЗПОЛЗВАНА ЛИТЕРАТУРА....8
ПРИЛОЖЕНИЯ....9
ПРАКТИЧЕСКА ЧАСТ....10

1. ВЪВЕДЕНИЕ

1.1. Задание за разработка

Настоящият дипломен проект има за цел разработването на уеб базирано приложение за музикален магазин с наименование MusicShop. Проектът е насочен към създаване на завършена информационна система, която да предоставя удобен начин за разглеждане на музикални продукти, работа с потребителски профили и извършване на онлайн поръчки. Приложението трябва да обединява публична клиентска част и административен модул, чрез който да се управляват основните данни в системата.

Разработеното решение е предназначено за потребители, които желаят да разглеждат и закупуват продукти, свързани с музиката, както и за администратори, които поддържат каталога и обработват постъпилите поръчки. В каталога са включени различни типове артикули, като музикални инструменти, албуми, аксесоари и мърчандайз продукти. Това налага системата да поддържа достатъчно гъвкава структура на данните и удобен интерфейс за работа с различни категории.

Заданието включва реализиране на основни функционалности за регистрация, вход и изход от системата, визуализация на продуктов каталог, търсене и филтриране на продукти, преглед на детайлна информация за отделен артикул, добавяне в списък с любими и създаване на поръчка. Освен това приложението трябва да предоставя секция с новини, чрез която да се публикува актуална информация, свързана с магазина и предлаганите продукти.

Съществена част от заданието е изграждането на административна зона. Чрез нея администраторът трябва да може да добавя, редактира и изтрива продукти, да публикува и управлява новини, както и да преглежда и обработва клиентските поръчки. По този начин MusicShop не е само представителен уеб сайт, а цялостна система, която покрива както потребителската, така и оперативната страна на един онлайн магазин.

От техническа гледна точка проектът се реализира чрез стандартни уеб технологии. Клиентската част е изградена с HTML, CSS и JavaScript, а сървърната логика е реализирана с PHP. За съхранение и управление на данните се използва MySQL/MariaDB. Комуникацията между интерфейса и сървърната част се осъществява чрез API крайни точки, които връщат данни в JSON формат.

1.2. Анализ на заданието

Анализът на заданието показва, че системата трябва да обслужва два основни типа потребители: обикновени клиенти и администратори. Клиентите имат нужда от лесен достъп до каталога, възможност за бързо ориентиране между различните категории и ясен процес за създаване на поръчка. Администраторите, от своя страна, се нуждаят от надежден механизъм за управление на продуктовата информация, новините и заявките от потребителите. Това налага ясно разграничаване на потребителските роли и контрол на достъпа до административните функции.

При анализа на публичната част се установява, че каталогът е централният елемент на системата. За да бъде полезен за крайния потребител, той трябва да предлага прегледна структура, възможност за филтриране по категория и бързо намиране на конкретен продукт. Поради разнообразието от артикули в магазина е важно данните да не се свеждат само до име и цена, а да включват и допълнителни характеристики като изпълнител, жанр, тип продукт, наличност, описание и изображение.

Процесът на поръчка е друг ключов елемент в системата. Той трябва да позволява на потребителя да избере желаните продукти, да прегледа съдържанието на количката и да въведе данни за доставка. От гледна точка на реализацията това означава, че системата трябва коректно да обработва множество артикули в една поръчка, да изчислява общата стойност и да записва информацията в свързани таблици в базата данни. Този процес трябва да бъде надежден, тъй като от него зависи коректността на основната бизнес логика.

Наличието на потребителски профили също е важен аспект от анализа. Регистрацията и входът в системата дават възможност за разграничаване на действията на различните потребители, за запазване на любими продукти и за проследяване на направени поръчки. Поддържането на сесии и ролеви модел е необходимо условие за създаване на по-завършено и реалистично уеб приложение.

В заданието присъства и изискване за новинарски модул. Макар тази функционалност да не е пряко свързана с извършването на продажби, тя е важна за изграждането на по-пълен облик на онлайн магазина. Чрез новините могат да се публикуват промоции, нови продукти и друга актуална информация, което подобрява комуникацията с потребителите.

От техническа гледна точка анализът показва нужда от ясно разделяне между потребителски интерфейс, сървърна логика и база данни. Такъв подход улеснява

поддръжката, тестването и бъдещото разширяване на системата. Използването на отделни API крайни точки за работа с продукти, поръчки, новини, регистрация, вход и любими позволява по-добра организация на кода и по-лесно проследяване на отговорностите в проекта.

1.3. Поставени цели

Основната цел на дипломния проект е създаването на функциониращо уеб приложение, което да симулира реална среда за работа на онлайн музикален магазин. За постигането на тази основна цел са поставени няколко конкретни подцели.

Първата цел е изграждане на удобен и интуитивен потребителски интерфейс, чрез който посетителите на сайта да могат лесно да разглеждат наличните продукти и да достигат до желаната информация без затруднение. Това включва ясна навигация, добре структурирани страници и последователна визуална организация.

Втората цел е реализиране на надеждна система за потребителска автентикация и управление на профили. Платформата трябва да поддържа регистрация на нови потребители, вход, изход и поддържане на активна сесия, като едновременно с това прави разлика между обикновен потребител и администратор.

Третата цел е разработване на функционален продуктов каталог с възможност за търсене, филтриране и визуализиране на подробна информация за всеки артикул. Каталогът трябва да представя различните продуктови категории по начин, който улеснява избора на потребителя.

Четвъртата цел е реализиране на механизъм за добавяне на продукти в любими и създаване на поръчки. Този модул трябва да позволява съхраняване на избрани артикули, попълване на данни за доставка и обработка на заявката до успешното ѝ записване в базата данни.

Петата цел е разработване на административен панел, чрез който да се поддържа съдържанието в системата. Администраторът трябва да има възможност да управлява продуктовия каталог, новинарската секция и направените поръчки, което е съществен елемент за практическата стойност на проекта.

Шестата цел е изграждане на добре структурирана база данни, която да осигурява надеждно съхранение на информацията за потребители, продукти, поръчки, любими и новини. Данните трябва да бъдат организирани така, че да позволяват коректни връзки между отделните модули на приложението.

Седмата цел е прилагане на добри практики при разработката, включително използване на подготвени SQL заявки, обработка на входните данни, ролеви контрол на

достъпа и разделяне на логиката между клиентска и сървърна част. По този начин проектът не само изпълнява поставените функционални изисквания, но и демонстрира добър инженерен подход при реализацията.

1.4. Очаквани резултати

Крайният очакван резултат от проекта е работещо уеб приложение, което позволява на потребителите да разглеждат музикални продукти, да се регистрират, да влизат в системата, да запазват любими артикули и да създават поръчки чрез удобен интерфейс. Приложението трябва да предоставя завършено потребителско изживяване, близко до това на реален онлайн магазин.

Очаква се системата да осигурява надеждна комуникация между клиентската част и сървърната логика, като действията на потребителя се обработват коректно и резултатите се отразяват в базата данни. В практически план това означава успешно записване на потребители, правилно извличане на продукти, създаване на поръчки и визуализиране на свързаната информация в отделните страници на приложението.

От административна гледна точка се очаква проектът да предлага работещ панел за управление, чрез който да се извършват основните операции по поддръжка на съдържанието. Това включва добавяне и редакция на продукти, управление на новини и преглед на поръчките, постъпили от клиентите.

Друг очакван резултат е проектът да демонстрира добро структуриране на кода и ясно разграничение между отделните компоненти на системата. Това е важно както за оценяването на дипломната работа, така и за възможността проектът да бъде доразвит в бъдеще.

В заключение, очакваният резултат не се изчерпва само с визуално завършен уеб сайт, а включва създаването на функционална, логически последователна и технически обоснована система, която показва умения за анализ, проектиране, реализация и документално описание на софтуерен продукт.

2. ОСНОВНА ЧАСТ

2.1. Избор на технологии

При разработването на MusicShop изборът на технологии е направен така, че да отговаря едновременно на функционалните изисквания на проекта, на практическите ограничения при реализацията и на целта дипломната работа да представя ясно разбираема и добре структурирана софтуерна система. Приложението трябва да обслужва няколко различни процеса: разглеждане на продуктов каталог, търсене и филтриране на артикули, регистрация и вход на потребители, поддържане на списък с любими продукти, създаване на поръчки, визуализация на новини и управление на съдържанието чрез административен панел. Това означава, че технологичният избор не може да бъде случаен, а трябва да осигурява добра свързаност между отделните модули и стабилна основа за цялото приложение.

При проекти от типа на уеб магазин е особено важно архитектурата да бъде логически разделена. Потребителят взаимодейства с интерфейса в браузъра, сървърът обработва заявките и прилага бизнес логиката, а базата данни съхранява информацията за продукти, потребители и поръчки. Ако тези три части не бъдат реализирани с подходящи средства, проектът бързо става труден за поддръжка и обяснение. Поради тази причина при MusicShop е използван класически уеб стек, при който всеки слой има ясно дефинирана отговорност и комуникацията между тях е лесна за проследяване.

Избраният технологичен стек включва HTML, CSS и JavaScript за клиентската част, PHP за сървърната логика и MariaDB/MySQL за съхранение и управление на данните. Този модел е добре познат в уеб разработката и има съществени предимства в контекста на дипломен проект. На първо място, той позволява бързо изграждане на работеща система без необходимост от сложна инфраструктура. На второ място, използваните технологии са достатъчно стандартни и широко разпространени, което прави проекта по-лесен за стартиране, тестване и защита. На трето място, този избор улеснява описването на архитектурата в документацията, тъй като връзките между клиент, сървър и база данни са ясно изразени.

При избора на технологии са взети предвид няколко основни критерия. Първият е съвместимостта с уеб стандартите и възможността приложението да работи в обичайна браузърна среда. Вторият е достъпността на инструментите за разработка и

липсата на нужда от тежък build процес или външни услуги. Третият е възможността проектът да бъде реализиран в локална среда и лесно демонстриран пред комисия. Четвъртият е добрата документация и широката общност около съответните технологии, което е важно както при самата разработка, така и при отстраняване на проблеми. Последният критерий е поддържането на инженерна яснота: проектът трябва да остане достатъчно разбираем, за да се покаже не само краен резултат, но и качествен процес по анализ и реализация.

Съществуват и други подходи за изграждане на подобна система. Възможно е например клиентската част да бъде реализирана чрез модерна JavaScript рамка като React, Vue или Angular, а сървърната част да се изгради с Node.js, Python или Java. Възможно е също така да се използва нерелационна база данни. В конкретния случай обаче подобни решения биха увеличили сложността, биха наложили повече конфигурация и не биха донесли съществено предимство спрямо поставените цели. MusicShop не изисква нито сложна клиентска state архитектура, нито микросървисен подход, нито съхранение на неструктурирани данни в големи мащаби. Поради това е предпочетено по-леко, но напълно професионално решение, което е по-подходящо за учебен проект с практическа насоченост.

Изборът на технологии влияе пряко и върху качеството на крайната система. Ако интерфейсът е изграден с подходящи уеб технологии, той може да бъде удобен, подреден и адаптивен. Ако сървърната логика е реализирана с правилните средства, бизнес процесите като вход, любими и поръчки ще бъдат обработвани надеждно. Ако базата данни е проектирана с правилен модел, информацията ще остане консистентна и лесна за извличане. Именно затова технологичният избор в MusicShop не трябва да се разглежда като списък от инструменти, а като основа, върху която стъпва цялостната работа на приложението.

2.1.1. Избор на програмен език

За клиентската част на MusicShop са избрани HTML, CSS и JavaScript. Макар HTML и CSS да не са програмни езици в тесния смисъл на думата, те са основните технологии за създаване на уеб интерфейс и заедно с JavaScript формират езиковата основа на потребителската част. Този избор е напълно естествен, тъй като приложението се използва през браузър и трябва да бъде достъпно без инсталиране на

допълнителен клиентски софтуер. Използването на стандартните уеб технологии гарантира съвместимост, бързо зареждане и възможност за работа в различни среди.

HTML е използван за изграждане на структурата на страниците в приложението. Чрез него са реализирани началната страница, каталогът, страницата за конкретен продукт, секцията с новини, страниците за вход и регистрация, любимите артикули, количката, поръчките и административният панел. Предимството на HTML е, че позволява съдържанието да бъде организирано в логически и семантични блокове, като заглавия, навигация, форми, карти за продукти, бутони и секции с данни. Това улеснява не само визуализацията, но и поддръжката на кода, защото структурата остава ясна и предвидима.

HTML е подходящ и защото е фундаментален стандарт на уеба. Той се поддържа от всички модерни браузъри и позволява еднакво представяне на основното съдържание на различни устройства. За MusicShop това е особено важно, тъй като системата трябва да бъде лесна за демонстрация и достъпна за крайния потребител. Освен това добрата HTML структура подпомага последващото стилизиране чрез CSS и добавянето на динамична логика чрез JavaScript. На практика HTML изпълнява ролята на основа, върху която се изгражда цялото потребителско изживяване.

CSS е избран за визуалното оформление на приложението. Неговата задача е да превърне структурираното HTML съдържание в завършен и удобен интерфейс. Чрез CSS в MusicShop се определят цветова схема, разположение на елементите, шрифтове, отстояния, размери на изображенията, бутони, продуктови карти, форми за въвеждане и таблици. Това позволява публичната и административната част да изглеждат като елементи от една обща система, а не като отделни несвързани страници.

Предимството на CSS е, че позволява ясното разделяне на съдържание и визия. Ако стиловете се намират в отделни файлове, те могат да бъдат променяни централизирано, без да се засяга логиката на страниците. Това е особено важно при проект като MusicShop, в който има множество изгледи с различно съдържание, но с нужда от последователна визуална идентичност. Подобен подход намалява дублирането на код, прави промените по-лесни и създава по-професионален резултат.

Още една причина за използването на CSS е възможността за адаптивен дизайн. Потребителите на онлайн магазини често разглеждат продукти и правят поръчки през различни устройства, включително телефони и таблети. Ако интерфейсът не се адаптира към различни размери на екрана, потребителското изживяване бързо се влошава. Чрез медийни заявки, гъвкави разпределения и относителни размери CSS

позволява страниците на MusicShop да останат удобни и четими независимо от устройството. Това е важно както от практическа, така и от проектна гледна точка.

JavaScript е избран като основен програмен език за динамичното поведение в браузъра. Той е необходим, защото модерното уеб приложение не трябва да се ограничава до статични страници. Потребителят очаква каталогът да може да се филтрира, продуктите да се зареждат динамично, любимите артикули да се маркират без пълно презареждане, а заявките към сървъра да се обработват бързо и плавно. Чрез JavaScript MusicShop реализира именно тази интерактивна страна на приложението.

В практическата реализация JavaScript се използва за извличане на данни от сървърните крайни точки, за визуализиране на продуктови списъци, за показване на подробна информация за конкретен артикул, за управление на количката, за работа с регистрацията и входа, както и за зареждане на новинарско съдържание. Това позволява приложението да бъде по-динамично и да реагира на действията на потребителя по-бързо. Важен плюс е, че логиката може да се изпълнява директно в браузъра, без да натоварва излишно сървъра за операции, свързани единствено с интерфейса.

Особено силна страна на JavaScript е възможността за асинхронна комуникация със сървъра чрез Fetch API. При MusicShop това позволява отделни части от интерфейса да се обновяват независимо една от друга. Например каталогът може да извлече филтриран списък с продукти, любимите могат да бъдат добавяни и премахвани чрез API заявка, а информацията за текущия потребител може да бъде заредена при отваряне на страницата. Всичко това се случва без задължително пълно презареждане, което прави приложението по-удобно и съвременно.

Използването на чист JavaScript, вместо рамка като React или Vue, е направено съзнателно. За проекти с много сложен интерфейс и голямо количество клиентска логика рамките имат съществени предимства, но MusicShop не попада в тази категория. Това е многостранично приложение, при което голяма част от логиката остава в сървъра и в добре определени API крайни точки. В такъв контекст vanilla JavaScript е напълно достатъчен. Той намалява зависимостите, не изисква компилация и прави кода по-пряк и по-лесен за разбиране в рамките на учебен проект.

Изборът на тези клиентски технологии има и образователна стойност. Чрез HTML, CSS и JavaScript се демонстрират фундаментални знания за работа с уеб страници, оформление и взаимодействие с потребителя. Вместо да се разчита изцяло на готови абстракции, разработката показва как реално се изграждат основните елементи

на един сайт и как те комуникират със сървърния слой. Това е важно за дипломна работа, защото показва не само умение да се използват готови инструменти, а и разбиране на базовите принципи на уеб разработката.

Следователно изборът на HTML, CSS и JavaScript е напълно обоснован. Тази комбинация осигурява надеждна и стандартизирана основа за клиентската част, позволява изграждане на удобен интерфейс, поддържа адаптивност и създава необходимата интерактивност. За MusicShop това е баланс между функционалност, техническа яснота и реалистичен обхват на разработката.

2.1.2. Избор на технология за сървъра

За сървърната част на MusicShop е избран PHP. Това е логичен избор за приложение от типа на онлайн магазин, защото PHP е технология, създадена именно за уеб среда и има много добра интеграция с HTML и релационни бази данни. Езикът е широко разпространен, поддържан от почти всички стандартни уеб среди и позволява сравнително бърза разработка на функционални приложения. В контекста на дипломен проект това е особено полезно, тъй като дава възможност да се постигне работещ резултат с умерена сложност и добра четимост на кода.

Силна страна на PHP е, че позволява лесно реализиране на сървърни API крайни точки. В MusicShop сървърната логика е организирана в директория api, където отделни файлове обработват различните операции: регистрация, вход, изход, извличане на продукти, работа с любими, създаване на поръчки, извличане на новини и административни действия. Тази структура е практична, защото всеки файл има конкретна отговорност и може лесно да бъде намерен, прегледан и тестван. Това също така улеснява документацията, тъй като архитектурата на сървъра може да бъде описана по модулен и ясен начин.

PHP е подходящ и заради вградената поддръжка на работа със сесии. В MusicShop това е ключово, защото приложението трябва да различава гост, обикновен потребител и администратор. След успешен вход в системата в сесията се записват идентификаторът и ролята на потребителя. По този начин останалите сървърни файлове могат да проверят дали дадено действие е позволено. Този подход е използван при достъп до любими артикули, история на поръчките и особено при

административните функции. Така технологията позволява сравнително лесна реализация на удостоверяване и контрол на достъпа.

Друга причина за избора на PHP е удобството при работа с HTTP заявки и отговори. Сървърът трябва да приема входни данни от формуляри или JSON заявки, да ги валидира, да изпълнява логика и да връща резултат в предвидим формат. PHP се справя добре с тази задача и позволява API крайните точки на MusicShop да комуникират с клиентската част чрез JSON. Това е достатъчно лек и ясен модел за обмен на данни, който не изисква допълнителен сложен framework, но е напълно адекватен за нуждите на проекта.

Важен аргумент в полза на PHP е и възможността лесно да се комбинират бизнес логика, проверки за сигурност и достъп до базата данни. При регистрация например сървърът валидира входните полета, проверява дали потребителското име или имейлът вече съществуват, хешира паролата и записва новия потребител. При вход проверява дали подадените данни съвпадат с наличния акаунт и създава сесия. При създаване на поръчка обработва списъка с артикули, изчислява стойностите и записва данните в няколко свързани таблици. Тези операции показват, че PHP е достатъчно гъвкав за съществената логика на системата.

В MusicShop са използвани и помощни механизми, които подобряват сигурността и подредеността на кода. Налице са функции за проверка дали потребителят е логнат, дали има администраторска роля, както и механизъм за работа с CSRF токени. Използвано е хеширане на пароли, а при основните заявки към базата данни се използват подготвени изрази. Това е важно, защото показва, че избраната сървърна технология не служи само за свързване на интерфейса с базата данни, а и за въвеждане на по-добри практики при работа с чувствителни данни и ограничаване на рискове.

PHP е подходящ и от организационна гледна точка. Проектът може да бъде стартиран локално в стандартна среда с уеб сървър и база данни, без да се изисква контейнеризация, отделна runtime конфигурация или компилация на сървърния код. Това улеснява както самата разработка, така и демонстрацията на готовото приложение. За дипломна работа е важно не само решението да бъде правилно от теоретична гледна точка, но и да може лесно да се представи и тества в реални условия.

Алтернативи като Node.js, Python или Java също могат да бъдат използвани за изграждане на сървърен слой. Те имат своите предимства, особено при мащабни системи или специализирани архитектури. В случая обаче MusicShop не изисква

подобна сложност. Основните нужди са да се обработват HTTP заявки, да се валидират данни, да се управляват потребителски сесии и да се осъществява връзка с релационна база данни. За този тип задачи PHP е напълно достатъчен и в много отношения по-практичен избор, защото намалява техническата тежест на проекта.

Практическото приложение на PHP се вижда най-ясно в модула за поръчки. При изпращане на поръчка сървърът приема JSON данни, проверява дали има логнат потребител, валидира списъка с артикули и данните за доставка, стартира транзакция, създава основния запис в таблицата orders, извлича цените на продуктите и записва редовете в таблицата order_items. След това обновява общата стойност и връща резултат към клиента. Това е типичен пример за добре реализирана сървърна бизнес логика, в която PHP изпълнява своята роля ефективно и последователно.

Следователно изборът на PHP като технология за сървъра е напълно оправдан. Той предоставя необходимите средства за реализиране на API, работа със сесии, контрол на достъпа, валидиране, сигурна обработка на данни и комуникация с базата данни. В рамките на MusicShop PHP осигурява баланс между практичност, разбираемост и достатъчна функционална мощност, което го прави много подходящ избор за проекта.

2.1.3. Избор на технология за базата данни

За съхранение и управление на данните в MusicShop е избрана релационната система MariaDB/MySQL. Това решение е пряко свързано с естеството на информацията, която приложението обработва. Онлайн магазинът работи с ясно дефинирани и взаимно свързани обекти: потребители, продукти, любими артикули, поръчки, редове към поръчки и новини. Тези данни имат стабилни зависимости и изискват надеждна структура, което прави релационния модел най-подходящ избор.

В MusicShop базата данни е организирана в отделни таблици, като всяка от тях отговаря на конкретен аспект от приложението. Таблицата users съхранява информация за потребителските акаунти, включително електронна поща, потребителско име, хеширана парола и роля. Таблицата products съдържа данни за артикулите в магазина, като име, категория, жанр, цена, наличност, описание и изображение, както и допълнителни полета за различните типове продукти. Таблицата favorites реализира връзката между потребител и продукт, а таблиците orders и order_items описват

поръчките и съдържанието им. Таблицата news съхранява публикациите за новинарския модул. Тази структура е ясна, логична и лесна за обяснение.

Основно предимство на MariaDB/MySQL е възможността да се дефинират първични ключове, уникални ограничения и външни ключове. В MusicShop това е особено важно, защото системата работи с взаимно свързани данни. Например всеки запис в order_items трябва да бъде свързан с валидна поръчка и валиден продукт, а таблицата favorites не трябва да допуска един и същ продукт да бъде добавян многократно за един и същ потребител. Чрез релационните ограничения тези правила се налагат директно на ниво база данни, което допринася за по-висока надеждност и намалява риска от грешки.

Използването на InnoDB като механизъм за съхранение носи друго важно предимство: поддръжка на транзакции. Това е критично за модула за поръчки. Когато потребителят създава поръчка, системата не извършва само едно действие, а последователност от няколко операции. Първо се записват общите данни за поръчката, след това се добавят всички свързани артикули, а накрая се обновява крайната цена. Ако в някой момент възникне грешка, е важно всички предишни стъпки да бъдат отменени, за да не останат частично записани данни. Точно това се постига чрез транзакционния модел на релационната база данни.

MariaDB/MySQL е много подходяща и поради добрата си интеграция с PHP. В проекта връзката към базата е централизирана в отделен конфигурационен файл, който използва mysqli и задава кодировка utf8mb4. Това е важно, защото приложението трябва коректно да работи с текстове на кирилица, със специални символи и с разнообразно съдържание. Подобна конфигурация създава добра основа за правилна обработка на потребителски имена, описания на продукти, новини и адреси за доставка.

Друга важна причина за избора на релационна база данни е възможността за ефективни SQL заявки. MusicShop трябва да поддържа извличане на продукти по категория, подкатегория, музикален жанр или текстово търсене, както и зареждане на поръчки за текущия потребител и обобщения за администраторския панел. SQL е много подходящ за подобни сценарии, защото позволява ясно формулиране на заявки, филтри, сортиране и join операции. Това прави работата с данните предвидима и сравнително лесна за оптимизиране.

В схемата на базата данни присъстват и индекси върху често използвани колони, както и full-text индекс за част от продуктовите полета. Това показва, че изборът на технология не е само теоретичен, а е свързан с реалната работа на приложението. При

по-голям каталог индексирането е важно, за да останат заявките достатъчно бързи. За проект като MusicShop това е ценен елемент, защото демонстрира, че структурата на базата данни е съобразена не само с логиката на данните, но и с практическата страна на извличането им.

Възможна алтернатива би била нерелационна база данни, например документно ориентирана система. Подобно решение може да бъде полезно при по-слабо структурирани данни или при приложения с много голям мащаб и хоризонтално разпределение. MusicShop обаче няма такива изисквания. Тук отношенията между обектите са ясни и съществени: един потребител може да има много поръчки, една поръчка съдържа много артикули, а един продукт може да присъства в любими на много различни потребители. Релационната база описва този модел по-естествено и по-надеждно.

MariaDB/MySQL е удачен избор и от гледна точка на администрацията и поддръжката. Технологията е широко разпространена, добре документирана и лесна за инсталиране в стандартна локална среда. Наличието на SQL dump на проекта допълнително улеснява възстановяването и демонстрацията на системата. Това е важно за дипломна работа, защото показва не само, че приложението работи, но и че данните му могат да бъдат организирани, прехвърляни и възстановявани по професионален начин.

В обобщение, изборът на MariaDB/MySQL е напълно обоснован от структурата на данните, нуждата от релационни връзки, необходимостта от транзакционност и добрата интеграция със сървърната технология. За MusicShop това решение осигурява стабилна и надеждна основа, върху която останалите модули на системата могат да функционират коректно. Релационната база данни не е просто място за съхранение, а важен компонент от цялостната архитектура на приложението.

2.2. Софтуерен дизайн

2.2.1. Етапи на разработка

Разработката на MusicShop преминава през последователни етапи, които осигуряват логичен преход от началната идея до работещо уеб приложение. Всеки етап има конкретна роля в оформянето на крайния резултат и не трябва да се разглежда изолирано. При създаването на уеб базирана система като музикален магазин е важно не само да се реализира визуална част, а и да се постигне правилно взаимодействие между интерфейс, сървърна логика и база данни. Именно затова етапите на разработка са разгледани като свързан процес, в който всяка следваща стъпка се основава на предходната.

В MusicShop тези етапи могат да бъдат обобщени в седем основни стъпки: събиране на информация и определяне на целите, планиране, дизайн на уеб приложението, кодиране и програмиране, оформяне на съдържанието и интерфейсните етикети, тестване и преглед, както и поддръжка и бъдещи актуализации. Тази структура следва логиката на реалната разработка и е подходяща за описване на проекта в дипломната документация.

1. Събиране на информация и поставяне на основни цели

Първият етап в разработката на MusicShop е свързан със събиране на необходимата информация и формулиране на основните цели на проекта. Това е изключително важна стъпка, защото именно тук се определя какъв проблем ще решава приложението, за кого ще бъде предназначено и какви функционалности трябва да бъдат реализирани. Без яснота по тези въпроси съществува риск разработката да се насочи към несвързани функции или към решения, които не носят реална стойност за крайния потребител.

В случая на MusicShop още в началото е било необходимо да се определи, че проектът няма да бъде само представителен сайт, а завършена уеб система за музикален магазин. Това означава, че тя трябва да обслужва както обикновени потребители, които разглеждат и поръчват продукти, така и администратор, който поддържа каталога, новините и поръчките. На този етап са уточнени и основните модули: продуктов

каталог, детайлни продуктови страници, потребителска регистрация и вход, любими артикули, количка, поръчки, новинарски модул и административна зона.

Събирането на информация включва и анализ на вида данни, с които ще работи системата. Тъй като магазинът е тематично насочен към музикални продукти, още в началото се установява, че каталогът няма да съдържа еднотипни артикули. В него ще присъстват музикални инструменти, албуми, мърч продукти и различни аксесоари. Това налага още на този ранен етап да се мисли за гъвкава структура на данните и за интерфейс, който да може да представя различни категории по ясен и последователен начин.

На етапа на поставяне на целите се формулира и инженерната рамка на проекта. Целта не е само визуално завършен сайт, а работещо приложение с реални сценарии на използване. Това означава, че трябва да има коректна регистрация на потребители, управление на сесии, комуникация между клиентската и сървърната част, запис и извличане на данни от база и възможност за администраторска поддръжка. Именно тази предварителна яснота позволява следващите етапи на разработката да бъдат по-структурирани и по-ефективни.

2. Планиране

След като са дефинирани основните цели, следва етапът на планиране. Тук идеята за проекта започва да се превръща в конкретна структура на приложение. Планирането включва определяне на отделните страници, техните връзки, основните модули и начина, по който потребителят ще се движи между тях. Това е етапът, в който се създава логиката на приложението още преди да е написан реалният код.

При MusicShop планирането започва с определяне на основните страници: начална страница, каталог, страница за единичен продукт, любими, количка, поръчки, вход, регистрация, новини и административен панел. Важно е всяка от тези страници да има ясно предназначение. Например каталогът трябва да представя много продукти и да поддържа филтриране, докато страницата за конкретен продукт трябва да акцентира върху подробната информация и действията за добавяне в любими или количка. По същия начин административният панел трябва да обединява операции по управление на данни, без да смесва излишно различни типове съдържание.

В етапа на планиране се вземат и решения за начина на организация на файловете. Това включва разделяне на клиентската част на HTML, CSS и JavaScript

файлове, отделяне на API файловете в самостоятелна директория и централизиране на връзката с базата данни. Подобна структура е важна не само за удобство при разработката, но и за бъдеща поддръжка. Ако кодът е подреден още от началото, разширяването на проекта и отстраняването на проблеми стават значително по-лесни.

Планирането включва и избор на подход за комуникация между отделните части на системата. За MusicShop е избрано многостранично уеб приложение, в което HTML страниците се зареждат отделно, а динамичните данни се извличат чрез асинхронни заявки към PHP API крайни точки. Това решение е важно още на етап планиране, защото влияе както върху клиентската логика, така и върху дизайна на сървърните файлове и върху начина на работа с базата данни.

3. Дизайн на уеб приложението

Третият етап е свързан със създаване на дизайна на уеб приложението. Тук вниманието се насочва към начина, по който системата ще изглежда и как ще бъде възприемана от потребителите. Дизайнът не се изчерпва само с избора на цветове и шрифтове, а включва и разположението на елементите, организацията на навигацията и общото потребителско изживяване. За онлайн магазин този етап е особено важен, защото интерфейсът трябва да бъде едновременно привлекателен и удобен за работа.

При MusicShop графичният дизайн е изграден около ясна и последователна структура. Почти всички страници съдържат общ хедър с навигация, статус на потребителя и основни действия, което създава усещане за единна система. Основното съдържание е разположено в отделни секции и карти, така че информацията да бъде лесно четима и добре разграничена. Това е особено важно при продуктовия каталог и при административната част, където потребителят трябва бързо да разпознава какво вижда и какво действие може да извърши.

На този етап се вземат решения и за визуалното разграничаване на отделните функции. Страниците за вход и регистрация са по-изчистени и ориентирани към формулярно взаимодействие. Каталогът е изграден около филтри и продуктови карти. Страницата за конкретен продукт акцентира върху изображението, описанието и цената. Количката и поръчките комбинират информационна и формална част. Административният панел използва по-функционален дизайн с форми, списъци и бутони за управление на съдържанието. Всичко това показва, че дизайнът на

приложението е съобразен с предназначението на всяка страница, а не е реализиран по един и същ шаблон навсякъде.

Съществен елемент от този етап е и предвиждането на адаптивност. В съвременната уеб среда потребителите достъпват сайтовете от различни устройства и размери на екрана. Поради това интерфейсът на MusicShop трябва да запази своята четимост и функционалност както на настолен компютър, така и на по-малки екрани. Този въпрос се взема предвид още на ниво дизайн, а не се оставя като последваща корекция.

4. Кодиране и програмиране

След като структурата и дизайна на приложението са ясни, започва етапът на кодиране и програмиране. Тук планираните идеи се превръщат в реален софтуерен продукт. За MusicShop този етап включва изграждане на HTML страниците, оформяне чрез CSS, добавяне на клиентска логика с JavaScript, създаване на сървърни крайни точки с PHP и изграждане на SQL схемата на базата данни. Това е най-обемният етап от разработката, тъй като тук се реализира същинската функционалност на системата.

При клиентската част се изграждат страниците, които потребителят реално вижда. В тях се добавят общи навигационни елементи, секции за съдържание, продуктови карти, форми за вход и регистрация, изгледи за количка, любими и поръчки, както и административни форми за управление. JavaScript логиката поема динамичните действия: зареждане на продукти и новини, филтриране, работа с localStorage количката, изпращане на данни към сървъра и визуално отразяване на резултатите от заявките.

Сървърната част се реализира чрез отделни PHP файлове, които играят ролята на API крайни точки. Това включва крайни точки за регистрация и вход, за извличане на продукти и любими, за създаване на поръчка, за работа с новини и за административни действия като добавяне, редактиране и изтриване на продукти. При реализацията на тези файлове е важно да се спазва ясно разделение на отговорностите, да се валидират входните данни и да се връщат коректни JSON отговори към клиента.

На същия етап се изгражда и базата данни, която съхранява постоянната информация за приложението. Създават се таблиците users, products, favorites, orders, order_items и news, определят се ключовете, индексите и връзките между тях. Кодиране

и програмиране в този контекст не означава само писане на интерфейсен код, а интегриране на всички слоеве на приложението в работеща система. Именно в този етап MusicShop преминава от проектна концепция към реално използваем софтуер.

5. Оформяне на съдържанието и етикетирането

След техническата реализация идва етапът на оформяне на съдържанието и етикетирането на интерфейса. Макар тази стъпка понякога да изглежда второстепенна, тя има пряко влияние върху използваемостта на приложението. Дори при добре реализирана логика, неясни заглавия, неудачни бутони и неразбираеми формулировки могат да затруднят потребителя и да направят системата по-трудна за използване.

В MusicShop този етап включва избор на подходящи наименования за страниците, бутоните, формулярните полета, секциите и съобщенията към потребителя. Например е важно бутоните за действия като „Добави в количката“, „Добави в любими“, „Запази продукт“ или „Добави новина“ да бъдат еднозначни и да не оставят съмнение какво ще се случи след натискането им. По същия начин е важно формулярите за вход, регистрация и поръчка да използват ясни етикети и логична последователност на полетата.

Оформянето на съдържанието е важно и заради тематичната цялост на магазина. Тъй като MusicShop е ориентиран към музикални продукти, текстовете трябва да поддържат тази идентичност и да звучат като част от една завършена система. Това се отнася както за заглавията на страниците и категориите, така и за описанията на продуктите, мета информацията и текстовете в административната част. Добре подбраните текстове правят приложението по-професионално и по-приятно за използване.

На този етап се оформят и съобщенията за грешки, потвърждение и статут на операциите. При успешно създаване на поръчка, при невалиден вход, при липса на логнат потребител или при проблем с дадена API операция, системата трябва да връща съобщения, които са разбираеми и полезни. Това е важна част от добрия потребителски интерфейс и пряко влияе върху цялостното възприятие за качеството на приложението.

6. Тестване, преглед и стартиране

Тестването е един от най-важните етапи в процеса на разработка, защото именно тук се проверява дали отделните компоненти на системата работят коректно както самостоятелно, така и заедно. При MusicShop този етап включва както проверка на отделни функционалности, така и проследяване на цели потребителски сценарии. Не е достатъчно например само да работи формата за вход; необходимо е да се провери и дали след успешен вход се появяват правилните елементи в интерфейса, дали се зарежда правилният статус на потребителя и дали достъпът до защитените секции е коректно ограничен.

При тестването на MusicShop се проверяват основни действия като регистрация на нов потребител, вход с валидни и невалидни данни, зареждане на каталога, филтриране и търсене на продукти, добавяне и премахване на любими, работа с количката, създаване на поръчка и извличане на история на поръчките. Отделно се проверява административната част: зареждане на всички поръчки, създаване и редакция на продукти, управление на новини и ограничаване на достъпа за неоторизирани потребители.

Прегледът на приложението включва не само функционални тестове, но и оценка на цялостната последователност на интерфейса. На този етап се забелязват дребни проблеми, които не винаги се виждат по време на самото кодиране: непоследователни надписи, неудобно разположени елементи, неясни съобщения или действия, които изискват прекалено много стъпки. Именно тук проектът се доизчиства и се доближава до по-завършен вид.

След успешното преминаване през тестовете системата може да се счита за готова за демонстрация и използване. В контекста на дипломния проект стартирането не означава непременно публичен деплой в интернет, а достигане до стабилно състояние, в което приложението може уверено да бъде представено, защитено и разглеждано като завършен софтуерен продукт.

7. Поддръжка и актуализации след стартиране

Последният етап не приключва с първоначалното завършване на приложението. Всяка система, особено уеб базирана, има нужда от поддръжка и потенциално развитие.

MusicShop също трябва да се разглежда като проект, който може да бъде надграждан. След началната реализация могат да бъдат добавени нови продуктови категории, по-богат административен контрол, плащания, по-подробни потребителски профили или разширени възможности за търсене и препоръки.

Поддръжката включва и отстраняване на открити проблеми, актуализация на данните, подобрения по сигурността и корекции при промяна на средата за изпълнение. Например промяна в версия на PHP, в конфигурацията на базата данни или в начина на хостване на приложението може да наложи допълнителни корекции. Ако кодът е структуриран добре още от предходните етапи, подобна поддръжка се извършва по-лесно и с по-малък риск за останалите модули.

За MusicShop възможностите за доразвитие са реални и логични. Приложението вече има основата на работещ онлайн магазин, което означава, че върху нея могат да се надграждат допълнителни функционалности. Точно затова е важно още на този етап да се подчертае, че разработката на софтуер не е еднократен акт, а процес, който може да продължи и след първоначалното завършване на проекта.

В заключение етапите на разработка на MusicShop показват подреден и аргументиран процес, започващ от анализ на идеята и достигащ до поддръжка и възможности за развитие. Всеки етап има своя принос към крайния резултат и заедно те формират цялостната рамка, в която проектът е реализиран. Именно тази последователност е една от причините приложението да бъде не просто набор от страници и файлове, а завършена уеб система с ясна логика и практическа стойност.

2.2.2. Софтуерна архитектура

Софтуерната архитектура на MusicShop е изградена върху ясен модел на разделение между клиентска част, сървърна логика и база данни. Този подход е особено подходящ за уеб приложение от типа на онлайн магазин, тъй като позволява различните функционалности да бъдат организирани в отделни слоеве с ясно определени отговорности. Потребителят работи с интерфейса в браузъра, сървърът обработва заявките и реализира бизнес логиката, а базата данни съхранява информацията за потребители, продукти, любими, поръчки и новини.

Клиентската част на приложението е реализирана чрез HTML, CSS и JavaScript. HTML страниците описват структурата на отделните изгледи, като начална страница, каталог, страница за продукт, любими, количка, поръчки, новини, вход, регистрация и административен панел. CSS отговаря за оформлението и визуалната последователност между страниците, а JavaScript добавя динамично поведение и осъществява комуникацията със сървърната част. По този начин интерфейсът не е просто статично съдържание, а работещ клиентски слой, който може да извлича данни, да обновява елементи и да реагира на действията на потребителя.

Сървърната част е реализирана чрез PHP и е организирана в директорията api. Вместо логиката да бъде събрана в един централен файл, приложението използва отделни крайни точки за различните операции. Например register.php и login.php обслужват автентикацията, products.php извлича продуктовия каталог, create_order.php обработва поръчките, get_news.php зарежда новините, а admin_add_product.php, admin_edit_product.php и admin_delete_product.php реализират административни действия върху продуктите. Тази организация е модулна и прави сървърната архитектура по-четима и по-лесна за поддръжка.

В рамките на сървърната архитектура важна роля играят и помощните файлове. db.php централизира връзката с базата данни и осигурява еднаква конфигурация за всички API файлове. _auth.php съдържа помощни функции за проверка на логнат потребител, администраторски достъп, метод на заявката и CSRF защита. admin_check.php е допълнителен механизъм за ограничаване на административните операции само до потребители с роля admin. Това показва, че архитектурата не е само набор от отделни файлове, а има и общи нива за повторна употреба на логика и контрол на достъпа.

От гледна точка на потребителския поток архитектурата на MusicShop следва предвидим модел. Потребителят извършва действие в браузъра, например избира филтри в каталога, добавя продукт в любими, попълва форма за регистрация или изпраща поръчка. JavaScript кодът изпраща HTTP заявка към съответната крайна точка в api слоя. PHP обработва заявката, валидира данните, комуникира с базата данни и връща резултат в JSON формат. След това клиентската част визуализира резултата в интерфейса. Това ясно разделение прави приложението лесно за проследяване и за документално описание.

Системата използва и сесии като част от архитектурния модел за удостоверяване и контрол на достъпа. След успешен вход в системата в сесията се съхраняват

идентификаторът на потребителя и неговата роля. Благодарение на това приложението може да различава гост, обикновен потребител и администратор. Това е от съществено значение за функционалности като любими артикули, история на поръчките и административен панел. Така архитектурата не само разпределя техническите роли между отделните слоеве, но и реализира логиката на достъпа в зависимост от типа потребител.

Базата данни е третият основен слой в архитектурата на приложението. Тя е изградена върху релационен модел и съхранява информацията в таблици като users, products, favorites, orders, order_items и news. Сървърната част комуникира с тези таблици чрез SQL заявки, а клиентската част никога не достъпва базата директно. Това е добро архитектурно решение, защото осигурява по-добра сигурност, по-ясен контрол на бизнес логиката и възможност за централизирана валидация на данните.

Особен елемент в архитектурата на MusicShop е използването на localStorage за количката. Това означава, че временното състояние на избраните продукти се пази от клиентската страна до момента, в който потребителят реши да завърши поръчката. Едва тогава данните се изпращат към сървъра и се записват в базата данни. Подобно решение намалява броя на ненужните заявки при промяна на количката и разделя временните клиентски данни от постоянните записи в системата. То е добър пример за прагматично архитектурно решение според нуждите на проекта.

Архитектурата на MusicShop може да се разглежда и като MVC-подобна структура, макар приложението да не използва формално цялостен MVC framework. Клиентските HTML страници и визуалните компоненти изпълняват ролята на изглед, PHP крайните точки обработват логиката и заявките като контролери, а базата данни и данните, които се извличат и записват, изпълняват ролята на модел. Този подход е достатъчно ясен за нуждите на проекта и позволява добра модулност, без да се въвежда излишна сложност.

В заключение софтуерната архитектура на MusicShop е лека, слоеста и добре съобразена с характера на приложението. Тя разделя ясно интерфейса, сървърната логика и данните, използва отделни API крайни точки за различните операции, поддържа сесии и ролев достъп и осигурява надеждна връзка с релационната база данни. Тази архитектура е достатъчно проста, за да бъде лесна за реализация и защита, но и достатъчно добре организирана, за да демонстрира стабилен инженерен подход при разработката на уеб приложение.

2.2.3. Описание и взаимодействие с приложението

Взаимодействието на потребителите с MusicShop е проектирано така, че приложението да бъде лесно за използване както от гости, така и от регистрирани потребители и администратори. Системата следва логика, при която основното съдържание е достъпно още при първо посещение, а допълнителните функционалности се отключват след регистрация и вход. Това е подходящо решение за уеб магазин, защото позволява на всеки посетител първо да разгледа продуктите и съдържанието, а едва след това да премине към по-ангажиращи действия като добавяне в любими, работа с количка и създаване на поръчка.

Когато потребителят отвори приложението, той попада на началната страница, от която може да достигне до продуктовия каталог, новините, страниците за вход и регистрация и останалите основни секции. Ако потребителят не е влязъл в профила си, той все пак може да разглежда наличните продукти, да използва филтрите в каталога, да отваря детайлна информация за отделен артикул и да се запознава с новинарското съдържание. По този начин системата остава функционална и полезна още на ниво гост-достъп, което е важен елемент от добрия потребителски поток.

След регистрация и успешен вход взаимодействието с приложението става по-богато. Потребителят може да добавя продукти в любими, да ги включва в количката и да създава поръчка чрез попълване на данни за доставка. Освен това има достъп до историята на собствените си поръчки, което прави системата по-близка до реален електронен магазин. Важен момент е, че приложението поддържа ясна граница между временни действия и окончателни операции. Например добавянето в количката се пази локално в браузъра, а реалното записване на поръчка в базата данни става едва след изрично потвърждение от страна на потребителя.

DATA FLOW ДИАГРАМА

Администраторът взаимодейства с приложението по различен начин. След вход с администраторска роля той получава достъп до административния панел, в който може да добавя, редактира и изтрива продукти, да управлява новини и да преглежда всички постъпили поръчки. Така системата не е ограничена само до клиентска функционалност, а предоставя и вътрешен модул за поддръжка на съдържанието. Това е особено важно за проекта, защото показва, че MusicShop е завършена информационна система, а не само публичен каталог.

От гледна точка на софтуерния дизайн взаимодействието с приложението следва предвидима и последователна структура. Потребителят извършва действие в интерфейса, клиентската част изпраща заявка към сървъра, сървърът обработва данните и връща резултат, който се визуализира обратно в браузъра. Именно тази схема стои зад регистрацията, входа, зареждането на продукти, управлението на любими, поръчките и административните действия. Благодарение на това MusicShop предлага ясно, подредено и лесно проследимо взаимодействие между различните потребители и отделните функционалности на системата.

2.2.4. Графичен дизайн и структура на приложението

След като са разгледани етапите на разработка, софтуерната архитектура и взаимодействието с приложението, следва да бъде представен графичният дизайн и общата структура на MusicShop. Този аспект е особено важен, защото определя по какъв начин потребителят възприема системата и доколко лесно може да се ориентира в нея. При онлайн магазин графичният дизайн няма само декоративна функция, а пряко подпомага навигацията, подреждането на съдържанието и потребителското изживяване.

MusicShop е изграден като многостранично уеб приложение, в което отделните страници имат ясно разграничени роли, но същевременно са обединени от общ визуален стил. Почти навсякъде присъстват хедър с лого и навигация, секция с основно съдържание и футър. Това създава усещане за последователност и прави приложението по-лесно за използване. Цветовата схема, разположението на елементите и повторението на общи компоненти са подбрани така, че да създадат единна идентичност на системата.

За да може MusicShop да функционира като завършен музикален магазин, приложението е изградено от няколко основни страници, всяка от които има конкретно предназначение и роля в потребителския поток. По-долу са разгледани най-важните страници и техният принос към общата структура на приложението.

Начална страница (Home / Landing Page)

Началната страница е първата точка на контакт между потребителя и приложението. Тя има представителна роля и служи като вход към останалите модули на системата. В нея се виждат логото на MusicShop, основната навигация и ключовите секции, които насочват потребителя към продуктовия каталог, новините и останалите функции на магазина. От гледна точка на графичния дизайн тази страница задава цялостния визуален тон на приложението и създава първото впечатление за неговия стил и тематика.

Структурно началната страница е организирана така, че още при първия преглед да се разбира какъв е характерът на системата. Водещи са представянето на магазина, акцентни секции за продукти или съдържание и ясни бутони за навигация. Това е важно, защото началната страница трябва не само да изглежда добре, но и да ориентира бързо потребителя какво може да прави в приложението.

Страница с продукти (Products Page)

Продуктовият каталог е една от най-съществените страници в MusicShop. Той е проектиран така, че да позволява лесно разглеждане на голям брой артикули чрез продуктови карти, филтри, сортиране и търсене. Визуално каталогът трябва да бъде ясен и балансиран, защото тук се съчетават много елементи: изображения, цени, категории, кратки описания и бутони за действие. При лоша структура тази страница би могла да стане претоварена, затова в MusicShop филтрите и резултатите са ясно разделени на отделни зони.

Продуктовите карти са ключов графичен елемент, тъй като представят най-важната информация в компактен вид. Всяка карта съдържа изображение, име, цена, мета информация и бутони за действия като преглед на детайли, добавяне в любими и добавяне в количката. Това позволява потребителят да преглежда каталога бързо и да преминава към следваща стъпка без излишни действия.

Страница за единичен продукт (Product Details Page)

Страницата за единичен продукт е естествено продължение на каталога и има за цел да покаже по-подробна информация за конкретен артикул. В графично отношение тя е проектирана така, че фокусът да пада върху изображението, името, цената и описанието на продукта. Допълнителната мета информация като категория, подкатегория, жанр, формат, размер или наличност е представена по структуриран начин, така че да допълва основните данни, без да отвлича вниманието от тях.

Тази страница е важна и от гледна точка на структурата на приложението, защото свързва каталога с действията на потребителя. Именно тук потребителят взема решение дали да добави продукта в любими или в количката. Поради това бутоните за действия са позиционирани ясно и логично в непосредствена близост до основната продуктова информация.

Страници за вход и регистрация (Login / Register)

Страниците за вход и регистрация са реализирани с по-изчистен графичен стил, защото при тях водещи са формите и полетата за въвеждане. Основната цел е потребителят бързо и без затруднение да създаде профил или да влезе в системата. Затова оформлението е по-семпло в сравнение с каталога и началната страница, но остава визуално съвместимо с останалата част от приложението.

Структурно тези страници имат важна роля, защото маркират прехода от гост-достъп към персонализирани функции. След успешен вход потребителят получава достъп до любими, поръчки и по-пълно взаимодействие със системата. Именно поради това страниците за вход и регистрация са не просто помощни екрани, а съществена част от общата структура на приложението.

Страница с любими (Favorites Page)

Страницата с любими е предназначена за регистрирани потребители и визуализира избраните от тях продукти в отделен списък. Графично тя е подобна на продуктовия каталог, но има по-персонализиран характер, защото съдържа само артикули, свързани с конкретния потребител. Това е полезно както от функционална, така и от структурна гледна точка, тъй като дава възможност за бърз преглед на предпочитаните продукти и улеснява последващото им добавяне в количката.

Страница с количка (Cart Page)

Количката е една от най-важните страници в търговската логика на приложението. В нея се комбинират информационна и формална част. От едната страна се показват избраните артикули, техните количества и общата стойност, а от другата страна се намира формата за попълване на данни за поръчката. Тази структура е особено подходяща, защото позволява на потребителя едновременно да следи съдържанието на количката и да подготвя изпращането на поръчката.

Графичният дизайн на тази страница е подчинен на яснота и подреденост. Важни са четимите редове за продуктите, бутоните за увеличаване и намаляване на количеството, общата сума и ясно разграничената форма за доставка. Това е необходимо, защото количката е преходен етап между разглеждането на продукти и завършването на покупката.

Страница с поръчки (Orders Page)

Страницата с поръчки позволява на потребителя да прегледа историята на вече направените заявки. В графично отношение тя трябва да представя информацията по структуриран и удобен за четене начин, като показва общи данни за поръчката и съдържащите се в нея артикули. Това е важен елемент от цялостната структура на приложението, защото добавя реалистичност и показва, че MusicShop не е само каталог, а система, която пази и организира търговска информация.

Страница с новини (News Page)

Новинарската секция допълва публичната част на приложението и има ролята да представя актуално съдържание, свързано с магазина, промоции или нови артикули. От гледна точка на графичния дизайн тази страница трябва да бъде достатъчно четима и структурирана, така че текстовото съдържание да не натоварва излишно потребителя. Това е и причината новините да бъдат отделени като самостоятелен модул, а не да бъдат смесени с продуктовия каталог.

Административен панел (Admin Dashboard)

Административният панел е специализирана страница, достъпна само за потребители с администраторска роля. Той предоставя централизирано управление на основните данни в системата: продукти, новини и поръчки. Визуално панелът използва същата обща идентичност като останалите страници, но е ориентиран по-силно към

функционалността. Това означава повече форми, по-компактни списъци, бутони за редакция и изтриване и ясна сегментация на отделните секции.

Структурно административният панел е особено важен, защото превръща MusicShop в пълноценна система за управление на съдържание, а не само в клиентски интерфейс. Именно тук се вижда практическата стойност на проекта като информационна система, която обслужва едновременно външната и вътрешната страна на един онлайн магазин.

В заключение графичният дизайн и структурата на MusicShop са подчинени на една основна цел: да осигурят ясно, последователно и удобно потребителско изживяване. Всяка страница има конкретна роля в общата логика на приложението, а повторението на общи визуални елементи създава усещане за завършен продукт. Това прави системата не само функционална, но и добре организирана от гледна точка на потребителския интерфейс и цялостната структура на приложението.

2.2.5. Дизайн на сървъра

Сървърната част на MusicShop е изградена върху PHP и MariaDB/MySQL и представлява основния логически слой на приложението. Тя обработва входните данни от клиентската част, комуникира с базата данни, управлява потребителските сесии и връща резултатите към интерфейса в JSON формат. За проект от типа на онлайн магазин това е особено важно, защото именно сървърът стои зад ключови операции като регистрация, вход, извличане на продукти, работа с любими, създаване на поръчки и административно управление на съдържанието.

Дизайнът на сървъра е организиран така, че всяка основна функция на системата да бъде реализирана в отделна крайна точка в директорията api. Това означава, че вместо цялата логика да бъде концентрирана в един голям файл, приложението използва набор от специализирани PHP файлове. Например register.php и login.php обслужват автентикацията, products.php и get_product.php работят с продуктовия каталог, add_favorite.php и remove_favorite.php реализират управлението на любими, create_order.php и get_orders.php обработват поръчките, а admin_add_product.php, admin_edit_product.php, admin_delete_product.php и admin_delete_order.php изпълняват административни действия. Подобна структура прави сървърния код по-четим и значително улеснява поддръжката.

Важна роля в дизайна на сървъра има и файлът db.php, който централизира връзката към базата данни. Вместо във всяка крайна точка да се настройват отново параметрите за достъп, те са изведени на едно място. Това е добро инженерно решение, защото при нужда от промяна на конфигурацията корекциите се правят само в един файл. Освен това по този начин всички API операции използват еднаква връзка и еднаква кодировка, което прави поведението на системата по-последователно.

Сървърният дизайн включва и механизми за удостоверяване и контрол на достъпа. След успешен вход в системата сървърът записва в PHP сесия идентификатора на потребителя и неговата роля. Това позволява на приложението да различава обикновен потребител от администратор. Файлове като _auth.php и admin_check.php реализират централизирани проверки дали потребителят е логнат, дали има администраторски права, дали използва правилния HTTP метод и дали е подаден валиден CSRF токен при чувствителни операции. Този подход е важен, защото защитава административните функции и намалява риска от неоторизиран достъп.

Друг ключов елемент в дизайна на сървъра е обработката на входните данни. При регистрация се валидират имейл, потребителско име и парола, при вход се проверяват подадените данни срещу базата, при работа с продукти се филтрират параметри като категория, търсене и сортиране, а при създаване на поръчка се валидират както избраните артикули, така и данните за доставка. Това показва, че сървърът не е просто посредник между интерфейса и базата данни, а реално изпълнява бизнес логиката на приложението.

Особено показателен за дизайна на сървъра е модулът за поръчки. При извикване на create_order.php сървърът първо проверява дали има активен потребител, след това валидира подадените артикули и задължителните полета, стартира транзакция, създава запис в orders, добавя редовете в order_items и накрая обновява крайната сума. Ако в който и да е момент възникне грешка, транзакцията се връща назад. Това е добър пример за сървърен дизайн, съобразен с реален търговски процес и с необходимостта от консистентност на данните.

В заключение сървърът на MusicShop е проектиран като лек, модулен и достатъчно сигурен слой, който обслужва както потребителските, така и административните операции в системата. Използването на отделни API крайни точки, централизирана връзка с базата данни, PHP сесии и допълнителни проверки за достъп правят сървърната част добре организирана и подходяща за проекта. Именно този

дизайн позволява приложението да работи надеждно и да поддържа всички основни функционалности на онлайн магазина.

2.2.6. Дизайн на базата данни

Уеб приложението MusicShop е изградено върху релационна база данни с име musicshop, която съхранява всички основни данни, необходими за работата на системата. Базата данни е проектирана така, че да обслужва както публичната клиентска част на магазина, така и административния панел. Тя трябва да осигурява надеждно съхранение на информация за потребители, продукти, любими артикули, поръчки и новини, като едновременно с това поддържа ясни връзки между отделните обекти и позволява изпълнение на коректни SQL заявки. Реалната структура на системата може да се види в SQL файла musicshop.sql, който отразява актуалното състояние на базата данни на проекта.

Базата данни на MusicShop е изградена от шест основни таблици: users, products, favorites, orders, order_items и news. Всяка от тях има отделна роля в общата логика на приложението и е проектирана така, че да поддържа конкретни функционалности. Релационният модел е избран, защото позволява ясна организация на структурирани данни и надеждно описване на връзките между потребители, продукти и поръчки. Това е особено важно за приложение от типа на онлайн магазин, в което една и съща информация се използва от различни модули и трябва да остане консистентна.

1. Таблица "users"

Таблицата users съхранява информация за всички регистрирани потребители на системата. Тя е основата на модула за регистрация, вход и контрол на достъпа. В нея се пазят идентификаторът на потребителя, неговото потребителско име, имейл адрес, хеширана парола, роля, допълнително поле is_admin и дата на създаване. Наличието на уникални ограничения върху полетата username и email гарантира, че системата няма да допуска дублиране на акаунти. Полето role се използва за разграничаване между обикновен потребител и администратор, което е важно за достъпа до административната част на приложението.

Структурата на тази таблица е сравнително компактна, но е от критично значение за цялата система. Чрез нея се реализират автентикацията, сесиите, връзката с любимите артикули и връзката с поръчките. В актуалния SQL файл се вижда, че в системата присъстват както администраторски, така и стандартни потребителски акаунти. Това показва, че таблицата не е само теоретично проектирана, а се използва активно в работеща среда с реални записи.

2. Таблица "products"

Таблицата products е централната таблица в базата данни, тъй като в нея се съхранява информацията за всички продукти в магазина. Тя съдържа основни атрибути като име, изпълнител, категория, подкатегория, цена, наличност, описание и изображение. Освен това са добавени и допълнителни полета като format, music_genre, size, color, merch_type, instrument_subtype и accessory_type. Причината за това е, че MusicShop работи с различни типове артикули и една обща продуктова таблица трябва да може да описва както музикални албуми, така и инструменти, мърч и аксесоари.

SQL файлът musicshop.sql ясно показва практическото приложение на този модел. В таблицата products присъстват записи за електрически и акустични китари, пиано, барабани, музикални албуми, мърч продукти и аксесоари. Вижда се също, че някои продукти използват локални изображения от проекта, а други външни URL адреси. Това потвърждава, че дизайнът на таблицата е достатъчно гъвкав да работи с различни типове данни и различни източници на съдържание, без да е необходимо разделяне на каталога в множество отделни продуктови таблици.

3. Таблица "favorites"

Таблицата favorites реализира връзката между потребителите и предпочитаните от тях продукти. Тя съдържа идентификатор на записа, user_id, product_id и дата на създаване. Това е междинна таблица, която реализира връзка много към много между users и products. Един потребител може да има много любими артикули, а един продукт може да бъде любим на много различни потребители. В актуалния SQL файл се вижда,

че таблицата действително съдържа записи за повече от един потребител, което потвърждава реалната употреба на този модул в системата.

Особено важно в дизайна на тази таблица е наличието на уникално ограничение върху комбинацията от user_id и product_id. Благодарение на него системата не допуска един и същ продукт да бъде добавен многократно в любими от един и същ потребител. Това е пример как ограничение на ниво база данни директно подпомага правилната работа на приложението и предотвратява дублиране на записи още преди да се достигне до логическа грешка в интерфейса или в сървърния код.

4. Таблица "orders"

Таблицата orders съхранява основната информация за направените поръчки. В нея се пазят идентификатор на поръчката, идентификатор на потребителя, крайна стойност, име на клиента, имейл, телефон, адрес за доставка, град, пощенски код, държава, бележка и дата на създаване. Тази структура е достатъчно подробна, за да покрие реален процес на създаване на поръчка, като позволява да се запази както връзката с регистриран потребител, така и цялата необходима информация за доставка. В актуалния dump се вижда, че таблицата съдържа реални тестови поръчки с попълнени данни за различни клиенти и адреси.

Връзката между orders и users е реализирана чрез външен ключ user_id. Той е конфигуриран така, че при изтриване на потребител стойността в поръчките да може да остане NULL, което е разумно решение, тъй като историята на поръчките не трябва да се губи. Таблицата orders не пази списъка с артикулите в поръчката, а само общата информация, което е правилен релационен подход и позволява по-добра нормализация на данните.

5. Таблица "order_items"

Таблицата order_items съдържа конкретните редове във всяка поръчка. В нея се пазят идентификатор на записа, идентификатор на поръчката, идентификатор на продукта, количество и единична цена. Това е стандартен модел за системи за

електронна търговия, защото позволява една поръчка да съдържа повече от един продукт и всеки артикул да бъде описан като отделен ред. Чрез този подход общата информация за поръчката остава в orders, а конкретното съдържание се пази в order_items.

Таблицата е свързана едновременно с orders и products чрез външни ключове. Това гарантира, че всеки ред в order_items ще сочи към валидна поръчка и към валиден продукт. От гледна точка на дизайна това е изключително важно, защото поддържа консистентност на данните и позволява лесно извличане на цялата информация за дадена поръчка чрез JOIN заявки. Наличието на редове с различни количества и различни единични цени в актуалния SQL файл показва, че моделът работи коректно в реални сценарии на поръчка с повече от един артикул.

6. Таблица "news"

Таблицата news съхранява новините, които се публикуват в публичната част на магазина. Тя съдържа идентификатор, заглавие, съдържание и дата на създаване. Макар тази таблица да е значително по-проста от останалите, тя има важна роля за допълващото съдържание на приложението. Благодарение на нея MusicShop не е ограничен само до каталог и поръчки, а включва и отделен информационен модул, който може да се управлява от администратора. В актуалния SQL файл се вижда, че системата вече съдържа реални новинарски записи, което потвърждава, че този модул е използван на практика.

Връзки между таблиците

Базата данни на MusicShop съдържа няколко основни връзки между таблиците. Първата е users -> favorites, която е връзка от тип едно към много, тъй като един потребител може да има повече от един любим продукт. От страната на products също съществува връзка едно към много към favorites, защото един продукт може да участва в много записи за любими. Заедно тези две връзки формират много към много

зависимост между потребители и продукти, реализирана чрез междинната таблица favorites.

Следващата важна връзка е users -> orders. Един потребител може да има повече от една поръчка, което прави тази зависимост връзка едно към много. Orders от своя страна е свързана с order_items, където връзката също е едно към много, защото една поръчка може да съдържа много редове с продукти. Таблицата products също има връзка към order_items, тъй като един продукт може да присъства в много различни поръчки. Именно тези връзки формират сърцевината на търговската логика на приложението. Таблицата news остава логически независима от останалите, тъй като обслужва отделен информационен модул.

Особености на дизайна

Една от основните особености на дизайна на базата данни е използването на обща таблица products за различни типове артикули. Това решение намалява сложността на системата и улеснява изграждането на общ каталог и административен панел. Вместо отделни таблици за музика, инструменти и мърч, всички продукти се управляват през един и същи модел, а специфичните характеристики се записват в допълнителни полета. Това е много практично решение за проект с учебна и демонстрационна цел и се потвърждава от съдържанието на musicshop.sql, където реално съществуват продукти от различни категории в рамките на една и съща таблица.

Друга важна особеност е наличието на индекси върху често използвани колони. В products има индекси върху category, genre и music_genre, а в orders и favorites има индекси върху user_id. Добавен е и full-text индекс върху name, artist и description в products, което подпомага текстовото търсене в каталога. Това показва, че базата данни не е проектирана само за съхранение на информация, а и за по-ефективно извличане на данни, което е особено важно за страница като каталога, където се комбинират филтриране, сортиране и търсене.

Системата е проектирана и с мисъл за сигурност и консистентност. Външните ключове гарантират референтна цялост, а използването на транзакции при създаване и изтриване на поръчки защитава данните от частично записване. Това е особено важно при операции, които засягат едновременно повече от една таблица. Така базата данни на MusicShop не е просто пасивно хранилище, а активна част от надеждната работа на приложението.

В заключение дизайнът на базата данни на MusicShop, разгледан през структурата в musicshop.sql, е изграден така, че да бъде едновременно ясен, логически последователен и достатъчно гъвкав за нуждите на проекта. Таблиците и връзките между тях отразяват основните процеси в онлайн магазина, а ограниченията, индексите и релационните зависимости създават стабилна основа за цялата система. Благодарение на този модел приложението може надеждно да съхранява, извлича и обработва информацията, необходима както за клиентската, така и за административната част на магазина.

2.2.7. CRUD заявки

В тази точка ще бъдат разгледани основните CRUD операции, които се използват в MusicShop. Съкращението CRUD идва от Create, Read, Update и Delete и описва четирите основни действия върху данните в една информационна система. В проекта тези операции са реализирани в различни модули, но най-ясно и пълно могат да бъдат проследени при управлението на продукти в административната част. Именно затова тук ще бъдат представени примери за създаване, четене, редактиране и изтриване на продуктови записи в базата данни.

Изборът на продуктовия модул не е случаен. Той дава възможност да се покаже цялостен път на една операция: от въвеждане на данни в административната форма, през изпращане на заявка от клиентската част, до обработка на данните в PHP сървъра и записването им в базата данни. По този начин CRUD операциите не се разглеждат само като SQL команди, а като завършен процес, в който участват интерфейсът, API слоят и релационният модел на системата.

CREATE заявка

Create операцията в MusicShop се използва за създаване на нов запис в базата данни. Един от най-показателните примери е добавянето на нов продукт от административния панел. Администраторът попълва формата с необходимите данни за артикула, като име, категория, цена, наличност, описание и изображение. В зависимост от типа на продукта могат да бъдат попълнени и допълнителни полета като музикален

жанр, формат, размер, цвят, подтип на инструмент или тип аксесоар. След това тези данни се събират в JavaScript обект и се изпращат към сървъра като JSON payload.

От страна на сървъра заявката се обработва от файла admin_add_product.php. В него първо се проверява дали са налични задължителните полета, след което се подготвя SQL INSERT заявка с параметри. Използването на подготвени изрази е важно, защото предпазва приложението от SQL инжекции и същевременно прави работата с различни типове входни данни по-надеждна. След успешно изпълнение на заявката новият продукт се записва в таблицата products и интерфейсът може да презареди списъка с налични артикули. По този начин Create операцията реализира създаване на нов обект в системата чрез съвместната работа на клиента, сървъра и базата данни.

CREATE ЗАЯВКА ЗА „PRODUCTS“ ЧРЕЗ PHP

```
$stmt = $conn->prepare("                    // Създава подготвена SQL заявка и я записва
   INSERT INTO products                     // Добавяме нов ред в таблицата products
   (name, artist, category, genre, price,   // Изброяват се колоните
    stock, description, image_url)
   VALUES (?,?,?,?,?,?,?,?)                 // Поставят се 8 placeholders (?) -  за всяка ст-ст
");

$stmt->bind_param(                          // Свързва PHP променливите с ? в SQL заявката
   "ssssddss",                              // Описва типовете данни
   $name,                                   // 1-вото ? ще получи стойността на $name
   $artist,                                 // 2-рото ? ще получи стойността на $artist
   $category,                               // 3-тото ? ще получи стойността на $category
   $genre,                                  // 4-тото ? ще получи стойността на $genre
   $price,                                  // 5-тото ? ще получи стойността на $price
   $stock,                                  // 6-тото ? ще получи стойността на $stock
   $description,                            // 7-мото ? ще получи стойността на $description
   $image_url                               // 8-мото ? ще получи стойността на $image_url
);

$stmt->execute();                           // Изпълнява заявката и записва данните в базата
```

READ заявка

Read операцията се използва за извличане и визуализиране на вече съществуващи записи от базата данни. В MusicShop добър пример за такава операция е зареждането на продуктовия каталог чрез файла products.php. Тази крайна точка извлича продуктите от таблицата products и връща информацията към клиентската част в JSON формат. Заявката поддържа и допълнителни филтри, като категория, подкатегория, музикален жанр, идентификатор на конкретен продукт, търсене по име и изпълнител и различни режими на сортиране.

Особено интересното при тази операция е, че тя не е ограничена до просто SELECT * FROM products. В зависимост от състоянието на потребителя и подадените параметри SQL заявката се разширява динамично. Ако в системата има логнат потребител, се прави и връзка с таблицата favorites, за да може в резултата да бъде отбелязано кои продукти вече са добавени в любими. Това показва, че Read операцията в реално приложение често е по-сложна от обикновено извличане на редове и може да комбинира логика от няколко таблици.

След изпълнение на заявката сървърът обхожда получените резултати и нормализира пътищата към изображенията. Това е допълнителна обработка, която прави върнатите данни по-подходящи за директно използване в интерфейса. Накрая клиентската част изгражда продуктовите карти и визуализира съдържанието на каталога. По този начин Read операцията осигурява не само достъп до данните, но и основата за визуалното представяне на каталога в MusicShop.

```
$sql = "SELECT p.id, p.name, p.artist,
p.category, p.price
        FROM products p
        WHERE p.category = ?
        ORDER BY p.price ASC";

$stmt = $conn->prepare($sql);
$stmt->bind_param("s", $category);
$stmt->execute();
$res = $stmt->get_result();
```

UPDATE заявка

Update операцията се използва за редактиране на вече съществуващ запис в базата данни. В MusicShop тя е реализирана в административния панел чрез възможността за редакция на продукт. Първо, администраторът избира кой продукт да бъде променен. След това системата зарежда текущите му данни и ги попълва в административната форма. Това позволява съществуващите стойности да бъдат прегледани и коригирани, без да се създава нов запис.

След като администраторът направи необходимите промени, клиентската част изпраща нов JSON payload към файла admin_edit_product.php. Сървърът извлича идентификатора на продукта и останалите стойности, след което изпълнява UPDATE заявка върху таблицата products. Подобно на Create операцията, и тук се използват подготвени SQL изрази с параметри. Това гарантира сигурност и улеснява работата с различните типове полета. След успешно изпълнение на заявката записът в базата данни се обновява, а административният интерфейс презарежда продуктовия списък с новите стойности.

Update операцията е важна, защото показва как една система не само създава и чете данни, но и ги поддържа актуални във времето. За онлайн магазин това е съществено, тъй като цени, наличности, описания и изображения често се променят. Без коректно реализирана Update логика приложението би загубило практическата си стойност като система за управление на съдържание.

МЯСТО ЗА CODE SNIPPET 3

Постави кодов фрагмент за Update операцията. Най-добре е да включиш: 1) логиката editProduct(id) от admin.html, която зарежда съществуващите данни в полетата; 2) частта, в която payload.id = editId и се прави fetch към /musicshop/api/admin_edit_product.php; 3) SQL UPDATE заявката от admin_edit_product.php с всички параметри и WHERE id=?. Това ще покаже пълния цикъл на редактиране на запис.

DELETE заявка

Delete операцията се използва за премахване на съществуващ запис от базата данни. В MusicShop пример за такава операция е изтриването на продукт от административния панел. Когато администраторът избере да премахне даден артикул,

системата първо показва потвърждение, за да се избегне случайно изтриване. След това към сървъра се изпраща заявка с идентификатора на съответния продукт.

Сървърната част обработва тази заявка чрез admin_delete_product.php. Във файла се проверява дали подаденият идентификатор е валиден, след което се изпълнява SQL команда DELETE FROM products WHERE id = ?. Ако има засегнат ред, операцията се счита за успешна и сървърът връща JSON отговор с ok=true. Ако продуктът не бъде намерен, се връща съобщение за грешка. Така Delete операцията е реализирана по контролиран начин и не позволява безусловно премахване на произволни данни.

Delete заявките са особено чувствителни в една информационна система, защото водят до необратима загуба на данни. Поради това те трябва да бъдат ограничени само до администраторски достъп и да използват ясна логика за валидация. Именно това се наблюдава в MusicShop. Изтриването е обвързано с административния панел, преминава през сървърна проверка и се изпълнява само при валиден идентификатор. По този начин Delete операцията е едновременно функционална и контролирана.

МЯСТО ЗА CODE SNIPPET 4

Постави кодов фрагмент за Delete операцията. На изображението трябва да се виждат: 1) JavaScript функцията deleteProduct(id) от admin.html с confirm прозореца; 2) fetch заявката към /musicshop/api/admin_delete_product.php?id=...; 3) PHP кодът от admin_delete_product.php, където се валидира id и се изпълнява DELETE FROM products WHERE id=?; 4) проверката на affected_rows. Така ще се покаже ясно как работи изтриването на запис.

От разгледаните примери се вижда, че CRUD операциите в MusicShop са реализирани като цялостен процес, а не само като отделни SQL команди. Във всяка операция участват потребителският интерфейс, клиентската JavaScript логика, сървърната PHP крайна точка и релационната база данни. Това е важно, защото показва как една информационна система на практика управлява своите данни и как различните слоеве си взаимодействат, за да предоставят завършена функционалност.

В заключение CRUD заявките са основен елемент от реализацията на MusicShop. Чрез тях системата може да създава нови записи, да извлича съществуваща информация, да редактира данните при нужда и да премахва вече ненужни записи. Именно благодарение на тези операции приложението функционира не само като визуален уебсайт, а като реална уеб базирана система за управление на музикален магазин.

2.3. Тестване

Върху приложението MusicShop е проведено тестване с цел да се провери неговата коректна работа, надеждност и съответствие с основните функционални изисквания на проекта. Тестването е важен етап от разработката, защото позволява да се установи дали отделните модули на системата работят правилно както самостоятелно, така и в комбинация помежду си. При уеб приложение от типа на онлайн магазин това е особено важно, тъй като грешка в един модул може да повлияе върху работата на регистрацията, каталога, количката, поръчките или административния панел.

При проверката на MusicShop са тествани както отделни потребителски сценарии, така и връзката между клиентската част, сървърната логика и базата данни. Извършена е проверка на регистрацията на нов потребител, входа в системата, зареждането на каталога, филтрирането и търсенето на продукти, добавянето и премахването на любими артикули, работата с количката и създаването на поръчки. Отделно е проверено дали историята на поръчките се визуализира коректно за текущия потребител и дали административният панел осигурява достъп само за потребители с администраторска роля.

Особено внимание е обърнато на административните функционалности, тъй като те засягат пряко съдържанието на системата. Тествани са добавянето, редакцията и изтриването на продукти, управлението на новини, прегледът на всички клиентски поръчки и административното изтриване на заявки. При тези проверки е проследено не само дали интерфейсът реагира правилно, но и дали съответните промени действително се отразяват в базата данни. По този начин е потвърдено, че връзката между клиентската част и API слоя работи коректно.

Отделно е проверена и работата на модулите, които използват сесии и контрол на достъпа. Проследено е дали при успешно логване системата разпознава ролята на потребителя, дали неоторизиран потребител няма достъп до административните действия и дали защитените операции връщат очакваното поведение. Това е важно, защото при приложение с различни типове потребители сигурността и правилното разделяне на правата са съществена част от качеството на реализацията.

При установяване на несъответствия или неудобства са направени корекции както в интерфейса, така и в сървърната логика. Това включва подобрения по валидацията на входните данни, по обработката на някои потребителски сценарии и по визуалното представяне на резултатите в отделните страници. След нанасяне на корекциите съответните тестове са повторени, за да се гарантира, че промените не са довели до нови проблеми в други части на системата.

В резултат от проведеното тестване може да се заключи, че MusicShop изпълнява основните изисквания на проекта и функционира като завършено уеб приложение за музикален магазин. Системата осигурява коректна работа на основните клиентски и административни модули, поддържа връзка с базата данни и демонстрира надеждна логика при работа с продукти, потребители, любими, поръчки и новини.

3. ЗАКЛЮЧЕНИЕ

3.1. Постигнати цели

Разработването на дипломния проект MusicShop доведе до създаването на цялостно уеб приложение, което реализира основните характеристики на съвременен онлайн магазин за музикални продукти. В процеса на работа беше изградено завършено решение, което обединява клиентска част, сървърна логика и база данни в една работеща система. По този начин първоначално поставената цел за създаване на функционално уеб приложение с реално практическо приложение беше изпълнена успешно.

Един от най-важните резултати от проекта е изграждането на удобен потребителски интерфейс, чрез който посетителите могат да разглеждат наличните продукти, да отварят подробна информация за тях, да ги добавят в любими или в количка и при необходимост да завършат процеса по поръчка. Структурата на приложението е организирана така, че да позволява лесна ориентация между отделните страници и бърз достъп до основните функции на системата. Това е съществено предимство, тъй като при онлайн магазин добрата навигация има пряко влияние върху удобството при използване.

Съществено постижение е и разработването на административната част на приложението. Чрез нея се осъществява управление на съдържанието в MusicShop, като администраторът има възможност да добавя нови продукти, да редактира вече съществуващи записи, да изтрива ненужни данни, да публикува новини и да следи постъпилите поръчки. Това превръща системата в динамична платформа, която може да бъде поддържана и актуализирана според конкретните нужди на магазина.

Постигната е и важната цел за реализиране на надеждна връзка между потребителския интерфейс, сървъра и базата данни. Чрез използваните програмни средства са разработени функционалности за създаване, извличане, редактиране и изтриване на информация, което осигурява коректната работа на всички основни модули. Проектираната база данни позволява структурирано съхраняване на данни за потребители, продукти, поръчки, любими артикули и новини, като между отделните

таблици са реализирани логически връзки, необходими за правилната работа на системата.

Работата по проекта има и съществена учебна стойност, тъй като в нея са приложени знания и умения, свързани с уеб програмиране, моделиране на база данни, софтуерен дизайн, работа с потребителски интерфейс и проверка на функционалността на готовото приложение. Изпълнението на всички етапи от разработката показва способност за преминаване от първоначална идея към практически реализиран софтуерен продукт.

3.2. Потенциално развитие

Въпреки постигнатите резултати, разработеното приложение дава възможност за бъдещо надграждане и усъвършенстване. Една от основните насоки за развитие е разширяване на потребителските функционалности чрез по-прецизни механизми за търсене и филтриране на продуктите по различни критерии като категория, цена, наличност и производител. Това би подобрило работата с каталога и би улеснило потребителите при откриването на желаните артикули.

Друга подходяща посока за развитие е обогатяване на профилите на регистрираните потребители. В бъдеща версия могат да бъдат добавени възможности за редакция на лични данни, разширена история на поръчките, система за оценки и коментари към продуктите, както и известия за промени в статуса на направените заявки. Подобни разширения биха доближили приложението до реалните изисквания на по-мащабни платформи за електронна търговия.

От особено значение е и потенциалното подобряване на сигурността и поддръжката на системата. В бъдеще могат да се добавят по-строги механизми за валидиране на входните данни, по-добра защита на достъпа до административните функции, по-надеждно управление на потребителските сесии и по-ясно структуриране на API слоя. Възможна е и интеграция с външни услуги, например системи за онлайн плащане, електронни известия и автоматизирана обработка на доставки.

В заключение може да се обобщи, че MusicShop представлява успешно реализиран дипломен проект, който съчетава теоретични знания и практическа реализация в областта на уеб технологиите. Създаденото приложение изпълнява

поставените цели, демонстрира функционална завършеност и създава стабилна основа за бъдещо развитие. Именно възможността за последващо надграждане прави проекта подходящ както за учебна разработка, така и като отправна точка за по-нататъшно усъвършенстване в сферата на електронната търговия.